অধ্যায় ১১

Error Handling, Debugging এবং Defensive Programming

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি JavaScript-এর Error Type চেনা, `try/catch/finally`, `throw`, Custom Error, Asynchronous Rejection, Browser DevTools, Breakpoints, Defensive Validation এবং User-Friendly Error Reporting ব্যবহার করতে পারবে। **Practical Project: OrderGuard — Order Validation Dashboard**।

১১.১ Error কেন গুরুত্বপূর্ণ?

Production Application-এ সব Input সঠিক হবে না, Network সবসময় সচল থাকবে না, এবং External Data নির্ভরযোগ্য নাও হতে পারে। একটি ভুলকে পুরো App Crash-এ পরিণত না করে **সুনির্দিষ্টভাবে শনাক্ত, ব্যাখ্যা ও নিয়ন্ত্রণ** করাই Error Handling। Error Handling কখনো Validation-এর বিকল্প নয়; বরং Validation-এর ব্যর্থতাকে নিরাপদভাবে উপস্থাপন করার উপায়।

তিনটি গুরুত্বপূর্ণ শ্রেণি:

1. **Syntax Error:** JavaScript Parse করার সময় ভুল; Code চালুর আগেই থেমে যেতে পারে।
2. **Runtime Error:** Code চলার সময় সমস্যা, যেমন `null.name` পড়া।
3. **Logical Error:** Code চলে, কিন্তু ফল ভুল; যেমন Discount ভুলভাবে হিসাব করা। Unit Tests এ ধরনের সমস্যা ধরতে সাহায্য করে।

```javascript
// Syntax error (শুধু উদাহরণ, চালাবে না): const = 42;
const user = null;
// console.log(user.name); // TypeError
const total = 100 - 10; // Discount 10% হলে এটি ভুল গণনা; সঠিক ফল 90, কিন্তু 1000 হলে একই formula ভুল
```

১১.২ Built-in Error Objects

`Error`, `TypeError`, `ReferenceError`, `RangeError`, `SyntaxError` প্রভৃতি Standard Error Object। `error.message` ব্যবহারকারীর উপযোগী Text নাও হতে পারে; `error.stack` মূলত Developer Diagnostics-এর জন্য।

```javascript
function readPrice(item) {
  if (!item || typeof item !== 'object') {
    throw new TypeError('Item must be an object');
  }
  if (typeof item.price !== 'number') {
    throw new TypeError('Item price must be a number');
  }
  return item.price;
}
```

`throw` দিয়ে যে কোনো JavaScript Value Throw করা যায়, কিন্তু **Error Instance** Throw করা উত্তম: Stack ও Error Type সহজে পাওয়া যায়।

১১.৩ `try`, `catch` এবং `finally`

```javascript
function safeParseJSON(text) {
  try {
    return { ok: true, value: JSON.parse(text) };
  } catch (error) {
    if (error instanceof SyntaxError) {
      return { ok: false, message: 'Invalid JSON format' };
    }
    throw error; // অপরিচিত Error চাপা দিচ্ছি না
  } finally {
    // প্রয়োজন হলে Cleanup বা Diagnostic Logging
  }
}
console.log(safeParseJSON('{"name":"Amina"}'));
console.log(safeParseJSON('{broken'));
```

`finally` সাধারণত `try/catch`-এর পরে চলে, এমনকি Return বা Throw থাকলেও। তবে `finally`-তে `return` দিলে আগের Return/Throw Override হতে পারে—এটি এড়িয়ে চলো।

১১.৪ Expected Failure বনাম Unexpected Failure

একটি Invalid Form Input **Expected Failure**: User-কে কোন Field সংশোধন করতে হবে বলো। কিন্তু `Cannot read properties of undefined` **Unexpected Failure**: Developer Log-এ Detail রাখো, User-কে সাধারণ বার্তা দেখাও। Sensitive Data, Access Token বা Password Console কিংবা Error Message-এ প্রকাশ করবে না।

```javascript
class ValidationError extends Error {
  constructor(message, issues = []) {
    super(message);
    this.name = 'ValidationError';
    this.issues = issues;
  }
}

function assertName(name) {
  if (typeof name !== 'string' || name.trim().length < 2) {
    throw new ValidationError('Invalid customer', ['Name needs at least 2 characters']);
  }
  return name.trim();
}
```

১১.৫ Defensive Programming: Trust Boundaries

Browser Input, URL Parameters, `localStorage`, Server Responses—সব External Data যাচাই করা প্রয়োজন। `typeof` যথেষ্ট নয়: `typeof null === 'object'`, `NaN`-এর Type-ও `number`। Numeric Input-এ `Number.isFinite()` বা `Number.isSafeInteger()` প্রয়োগ করো।

```javascript
function parsePositiveInteger(input) {
  const text = String(input).trim();
  if (!/^[1-9]\d*$/.test(text)) return null;
  const value = Number(text);
  return Number.isSafeInteger(value) ? value : null;
}
console.log(parsePositiveInteger('12'));   // 12
console.log(parsePositiveInteger('12px')); // null
console.log(parsePositiveInteger('1.5'));  // null
```

`parseInt('12px', 10)` কিন্তু `12` ফেরত দেয়। Strict Form Validation-এ এটি প্রায়ই অনাকাঙ্ক্ষিত। আবার অতিরিক্ত Validation-ও ক্ষতিকর হতে পারে; Business Rules অনুযায়ী পরিষ্কার Constraint নির্ধারণ করো।

১১.৬ Async Error Handling

```javascript
async function getUser(id, signal) {
  const response = await fetch(`/api/users/${encodeURIComponent(id)}`, { signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
async function loadUser(id) {
  const controller = new AbortController();
  try {
    return await getUser(id, controller.signal);
  } catch (error) {
    if (error?.name === 'AbortError') return null;
    console.error('User request failed', error);
    throw error;
  }
}
```

মনে রাখবে: `fetch()` সাধারণত HTTP 404/500-এ Promise Reject করে না; `response.ok` পরীক্ষা করতে হয়। কিন্তু Network Failure-এ Reject হতে পারে। `await` ছাড়া Promise Return করলে Outer `try/catch` সেটির Rejection ধরতে নাও পারে; Promise `await` করো বা Caller-কে Handling-এর দায়িত্ব দাও।

১১.৭ Debugging with Browser DevTools

1. **Console:** `console.log`, `console.table`, `console.error` দিয়ে State পরীক্ষা করো। Production-এ Sensitive Data Log করবে না।
2. **Sources → Breakpoint:** নির্দিষ্ট Line-এ Execution Pause করে Variables ও Call Stack দেখো।
3. **Step Over / Into / Out:** Function Flow অনুসরণ করো।
4. **Network:** Request URL, Status, Timing, Payload ও Response দেখো।
5. **Preserve Log:** Navigation বা Reload-এর পরও Debug History রাখতে সাহায্য করে।
6. **Reproduce → Isolate → Fix → Test:** Bug Report-এর নির্ভরযোগ্য Workflow।

```javascript
function discountedTotal(price, discountRate) {
  debugger; // DevTools খোলা থাকলে এখানে Pause করতে পারে
  return price * (1 - discountRate);
}
// discountedTotal(1000, 0.1) => 900
```

১১.৮ Error Handling Anti-Patterns

```javascript
// ❌ Error গিলে ফেলা
try { JSON.parse('{broken'); } catch (error) {}

// ❌ কোনো Context ছাড়াই generic error
// throw new Error('Oops');

// ❌ User Input HTML হিসেবে Inject করা
// element.innerHTML = userInput;

// ✅ নিরাপদভাবে Text Render করা
// element.textContent = userInput;
```

কোনো Error Retry করা উচিত কি না, সেটিও গুরুত্বপূর্ণ: Validation Error Retry করলে লাভ নেই; সাময়িক Network Error-এ সীমিত Backoff সহ Retry করা যায়। Payment বা Booking-এর মতো Side Effecting Request Retry করতে হলে Server-side Idempotency প্রয়োজন।

১১.৯ Practical Project — OrderGuard: Order Validation Dashboard

Goal: Customer Name, Email, Product ও Quantity যাচাই করে Order Total দেখানো; Invalid Input-এ স্পষ্ট Error, Unexpected Error-এ Safe Fallback। এটি **Learning Demo**, কোনো Payment/Backend নেই।

Project Structure

```
source-code/chapter-11/
├── index.html
├── style.css
├── app.mjs
├── order-core.mjs
└── order-core.test.mjs
```

Step 1 — Domain Logic

`order-core.mjs`-এ `ValidationError`, `parseQuantity`, `validateOrder` এবং `formatBDT` রয়েছে। Quantity 1–100, Product Catalog-এ উপস্থিত এবং Total Safe Integer হতে হবে। `Object.hasOwn()` ব্যবহার করা হয়েছে, যাতে Prototype Property ভুল করে Product হিসেবে গৃহীত না হয়।

Step 2 — User Interface

`index.html` Form, Feedback Region (`role="status"`) এবং Accepted Order List তৈরি করে। `app.mjs`-এ Submit Handler `try/catch/finally` ব্যবহার করে; `finally` Button পুনরায় Enable করে। DOM-এ `textContent` ব্যবহারে User Text HTML হিসেবে Execute হয় না।

Step 3 — Run and Test

```
TERMINAL
# project folder-এর ভেতর থেকে
python3 -m http.server 8000
# browser: http://localhost:8000/

# Node.js 20+ recommended
node --test order-core.test.mjs
```

Expected Behavior: `Amina`, `amina@example.com`, `Keyboard`, Quantity `2` দিলে Total BDT 2,500। Quantity `0`, Unknown Product, Invalid Email দিলে Validation Message। Order History শুধুমাত্র চলমান Page Session-এর Memory-তে থাকবে; Refresh করলে মুছে যাবে।

Step 4 — Manual QA Checklist

- ☐ Valid Order Submit করলে Success Message এবং History Row দেখা যায়।
- ☐ Invalid Name, Email, Quantity আলাদাভাবে Reject হয়।
- ☐ Double Submit এড়াতে Processing চলাকালে Button Disabled থাকে।
- ☐ Error-এর পর Button পুনরায় Enabled হয়।
- ☐ HTML-like Customer Name Text হিসেবেই Render হয়।
- ☐ Mobile Width-এ Form ব্যবহারযোগ্য।

১১.১০ Testing ও Error Diagnosis

Automated Tests-এ Happy Path-এর পাশাপাশি Boundary ও Failure Cases পরীক্ষা করো। যেমন: Quantity `0`, `101`, `1.5`, `1e2`, `null`; Unknown Product; Empty Name; Malformed Email; Unsafe Total। UI Tests আলাদা, কারণ Node Unit Test DOM Render যাচাই করে না।

```
JAVASCRIPT
import assert from 'node:assert/strict';
import { validateOrder, ValidationError } from './order-core.mjs';
const catalog = { pen: { name: 'Pen', price: 20 } };
assert.throws(() => validateOrder({
  customer: 'Amina', email: 'amina@example.com',
  productId: 'pen', quantity: 0
}, catalog), ValidationError);
```

১১.১১ Exercises

1. `safeDivide(a,b)` লেখো: Invalid Input ও Division by Zero-তে Error Throw করবে।
2. `safeJSONParse(text)` Result Object (`ok`, `value` / `message`) Return করবে।
3. Custom `ValidationError` তৈরি করে Invalid Username Reject করো।
4. `fetch()` Response-এ `response.ok` যাচাই করার Function লেখো।
5. `parseQuantity()`-তে `0`, `1.5`, `101`, `1e2` Reject হচ্ছে কি না Test করো।

6. `finally` ব্যবহার করে Async Submit-এর Loading State Reset করো।
7. DevTools-এ Breakpoint বসিয়ে `validateOrder()`-এর `issues` Array পরীক্ষা করো।
8. DOM Rendering-এ `innerHTML` বদলে `textContent` ব্যবহারের Security Benefit ব্যাখ্যা করো।
9. `Object.hasOwn()` ছাড়া Catalog Lookup-এ কী সমস্যা হতে পারে দেখাও।
10. OrderGuard-এর জন্য তিনটি নতুন Unit Test এবং দুটি Manual UI Test Case লিখো।

১১.১২ Exercise Solutions

১.

```javascript
function safeDivide(a, b) {
  if (!Number.isFinite(a) || !Number.isFinite(b)) throw new TypeError('Numbers required');
  if (b === 0) throw new RangeError('Cannot divide by zero');
  return a / b;
}
```

২.

```javascript
function safeJSONParse(text) {
  try { return {ok: true, value: JSON.parse(text)}; }
  catch (error) { if (error instanceof SyntaxError) return {ok: false, message: 'Invalid JSON'};
throw error; }
}
```

৩.

```javascript
class ValidationError extends Error {
  constructor(message) { super(message); this.name = 'ValidationError'; }
}
function validateUsername(name) {
  if (typeof name !== 'string' || name.trim().length < 3) throw new ValidationError('Minimum 3
characters');
  return name.trim();
}
```

৪.

```javascript
async function fetchJSON(url) {
  const response = await fetch(url);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
```

৫. `parseQuantity()`-এ Strict Digit Regex, `Number.isSafeInteger()` এবং `<= 100` Constraint ব্যবহার করো; Project-এর Unit Tests-এ সব Cases আছে।

৬.

```javascript
async function submitOrder(button, task) {
  button.disabled = true;
  try { return await task(); }
  finally { button.disabled = false; }
}
```

৭. DevTools Sources Tab-এ `validateOrder()`-এর `if (issues.length)` Line-এ Breakpoint দাও; Invalid Email Submit করে `issues` Inspect করো।

৮. `textContent` Input-কে Plain Text হিসেবে দেখায়; `innerHTML` Markup Parse করে এবং Untrusted Input-এ XSS ঝুঁকি তৈরি করতে পারে।

৯. Prototype Chain-এর `toString` বা `__proto__`-এর মতো Property Catalog Product নয়; `Object.hasOwn(catalog, id)` দিয়ে নিজস্ব Key নিশ্চিত করো।

১০. Unit Test: Empty Name, Unknown Product, Unsafe Total। Manual UI: Submit Feedback, Responsive Layout।

১১.১৩ MCQ Quiz — ১০টি প্রশ্ন

1. `try/catch` মূলত কোন Error Handle করে? (A) Syntax-only (B) Runtime thrown errors (C) CSS errors (D) HTTP automatically
2. `finally` কখন চলে? (A) শুধু Success (B) শুধু Failure (C) সাধারণত উভয় ক্ষেত্রেই (D) কখনো নয়
3. `typeof null` কী? (A) null (B) undefined (C) object (D) boolean
4. `fetch()` HTTP 404 পেলে সাধারণত কী হয়? (A) Promise Reject (B) Promise Resolve, `ok=false` (C) Page Reload (D) Browser Crash
5. `Number.isSafeInteger(1.5)` কী? (A) true (B) false (C) null (D) Error
6. `throw new TypeError()` কী করে? (A) Log Only (B) Exception Throw (C) Promise Resolve (D) DOM Update
7. Untrusted User Text Render-এর জন্য কোনটি নিরাপদ? (A) `innerHTML` (B) `eval` (C) `textContent` (D) `document.write`
8. `catch`-এ Unknown Error এলে কী করা ভালো? (A) Silent Ignore (B) Context অনুযায়ী Log ও Rethrow/Handle (C) Password Print (D) Infinite Retry
9. Debugger Breakpoint-এর উদ্দেশ্য? (A) CSS Style (B) Execution Pause ও State Inspect (C) Network Speed (D) Build Bundle
10. `Object.hasOwn(catalog,id)` কেন? (A) Async Support (B) Own Property Check (C) Price Format (D) Browser Cache

Answer Key ও ব্যাখ্যা

1. **B** — Throw হওয়া Runtime Error Catch করা যায়; Parse-time Syntax Error সাধারণ Script-এ আগে ঘটে।

2. **C** — `finally` Cleanup-এর জন্য, Success/Failure দুই পথেই সাধারণত চলে।
3. **C** — JavaScript-এর ঐতিহাসিক আচরণ।
4. **B** — HTTP Status পরীক্ষা করতে `response.ok` প্রয়োজন।
5. **B** — Safe Integer হতে পূর্ণসংখ্যা হতে হবে।
6. **B** — Execution-এর Normal Flow ভেঙে Exception তৈরি করে।
7. **C** — Plain Text Rendering, Markup Interpretation নয়।
8. **B** — Unknown Error গোপন করলে Debugging কঠিন হয়।
9. **B** — Call Stack, Variables ও Flow Inspect করা যায়।
10. **B** — Inherited Property-কে Catalog Entry হিসেবে গ্রহণ ঠেকায়।

১১.১৪ Chapter Recap এবং পরবর্তী অধ্যায়

Error Handling মানে শুধু `try/catch` নয়: Input Boundary যাচাই, User-Friendly Feedback, Safe Logging, Tests এবং Debugging Process একসঙ্গে Reliability তৈরি করে। এই অধ্যায়ের OrderGuard Project-এ Pure Validation Logic ও DOM Interaction আলাদা রাখা হয়েছে—পরবর্তী অধ্যায়ে **Modern JavaScript, ES Modules এবং Clean Code** শেখার জন্য এটি ভালো ভিত্তি।

পরবর্তী অধ্যায়: ১২ — Modern JavaScript (ES6+), Modules এবং Clean Code Practices।